CHAPTER 4

GETTING STARTED WITH C++

CHAPTER ESSENCE AT A GLANCE:
C++ merges high-level structured abstraction with efficient low-level hardware control, organizing source logic into lexical tokens, preprocessor directives, and standard library input-output streams.

4 CORE PRINCIPLES TO UNDERSTAND FIRST

• **Lineage and Paradigm Hybridization:** Developed by Bjarne Stroustrup at Bell Laboratories, C++ enhances procedural C with object-oriented classes, data abstraction, and strong type safety while retaining machine-level execution speed.

• **Lexical Token Classification:** The compiler partitions source code into atomic tokens: keywords with reserved compiler meanings, user-defined identifiers, constant literals, computational operators, and punctuators.

• **Preprocessor Orchestration and Directives:** Preprocessor statements commencing with the hash symbol ingest header files, define macros, and configure compilation parameters prior to lexical analysis.

• **Standard Stream Architecture:** The standard library handles input and output as sequential byte streams, coupling the keyboard to standard input stream cin and the screen to standard output stream cout.

ESSENTIAL ARCHITECTURE & CORE SYNTAX CARDS

$$\text{Lexical Tokens} = \text{Keywords} + \text{Identifiers} + \text{Literals} + \text{Operators} + \text{Punctuators}$$

Five Primary Lexical Token Categories in C++

$$\text{Valid Identifier} \in \{\text{[A-Za-z][A-Za-z0-9]}^*\} \setminus \{\text{C++ Reserved Keywords}\}$$

Identifier Syntactic Grammar Rules

$$\text{Program Structure} = \textbf{int main}() \rightarrow \{\text{Declarations} \rightarrow \text{Statements} \rightarrow \textbf{return } 0;\}$$

Execution Entry Point and Minimal Function Body

$$\text{Preprocessor Pipeline} = \text{Source (.cpp)} \xrightarrow{\#\text{include <iostream>}} \text{Expanded Unit} \xrightarrow{\text{Compiler}} \text{Object Code}$$

Source Preprocessing and Header Inclusion Cycle

CRUCIAL EXAM PRECAUTIONS & VALUATION RUBRIC SECRETS

• **Alert:** Identifiers cannot begin with a numeric digit and cannot contain spaces or special punctuation characters.

• **Alert:** C++ is strictly case-sensitive; main, Main, and MAIN are treated as completely different identifiers.

• **Alert:** Preprocessor directives like #include must never terminate with a semicolon.

HIGH-YIELD CENTUM QUESTIONS & STEP-BY-STEP EXPLANATIONS

1. What are tokens in C++? List the five distinct categories of lexical tokens. [Score: 2 Marks]

Answer & Key Valuation Steps:

• Token Definition: The smallest individual units in a C++ program that are recognized by the compiler during lexical analysis. **[1 Mark]**

• Five Token Categories: Keywords (reserved words), Identifiers (names of variables and functions), Literals (constants), Operators, and Punctuators (delimiters). **[1 Mark]**

2. Identify which of the following identifiers are valid, and give reasons for the invalid ones: totalMarks, 3rdRank, switch, stuName. [Score: 2 Marks]

Answer & Key Valuation Steps:

• totalMarks: Valid (starts with an alphabet and contains only alphanumeric characters). **[0.5 Mark]**

• 3rdRank: Invalid (identifiers cannot begin with a numeric digit). **[0.5 Mark]**

• switch: Invalid (switch is a reserved C++ keyword). **[0.5 Mark]**

• stuName: Valid (starts with an alphabet and contains only letters). **[0.5 Mark]**

3. Explain the role of `#include <iostream>` and `using namespace std;` in a C++ program. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Role of #include <iostream>: Preprocessor directive that instructs the compiler to incorporate the standard input-output stream header file providing declarations for cin and cout. **[1.5 Marks]**

• Role of using namespace std: Declares that all identifiers defined within the standard namespace (such as cin, cout, endl) can be used directly without the explicit std:: prefix. **[1.5 Marks]**

4. Distinguish between character literals and string literals in C++ with examples. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Character Literals: Single characters enclosed within single quotation marks (e.g. 'A', '7', '\
'); allocated exactly 1 byte of memory in standard char type. **[1.5 Marks]**

• String Literals: Sequence of zero or more characters enclosed within double quotation marks (e.g. "KERALA", "C++"); automatically appended with a terminating null character '\0' consuming length plus 1 bytes. **[1.5 Marks]**

5. Write a complete C++ program to display your school name, class, and roll number on separate lines. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Program Implementation: #include <iostream>\
using namespace std;\
int main() {\
 cout << "School: Model Higher Secondary School" << endl;\
 cout << "Class: Plus One Commerce" << endl;\
 cout << "Roll Number: 18" << endl;\
 return 0;\
} **[2 Marks]**

• Explanation: cout streams text to the screen; endl inserts a newline character after each string. **[1 Mark]**

6. Explain escape sequences in C++. State the specific action of `\<br/>`, `t`, `\\`, and `\"`. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Escape Sequences: Non-printable or formatting characters preceded by a backslash that convey special printing instructions to output devices. **[1 Mark]**

• \
: Newline character (advances cursor to the beginning of the next line). t: Horizontal tab (moves cursor forward to the next tab stop). **[1 Mark]**

• \\: Backslash (displays a literal backslash character). \": Double quote (prints a literal double quotation mark inside a string). **[1 Mark]**

7. Describe the complete compilation and execution workflow of a C++ source program. **[Score: 4 Marks]**

Answer & Key Valuation Steps:

• Source Code Authoring: The programmer writes human-readable instructions in a text editor and saves the file with .cpp extension. **[1 Mark]**

• Preprocessing: The preprocessor handles directives (like #include) and expands macros, producing an intermediate compilation unit. **[1 Mark]**

• Compilation: The compiler analyzes syntax, reports errors, and translates valid source code into machine-level object code (.obj or .o). **[1 Mark]**

• Linking and Execution: The linker combines object code with runtime standard library functions into an executable binary (.exe), which the operating system loader places into RAM for execution. **[1 Mark]**